Theory Questions :-

==========

Q1. What are real-life applications of stack?
Q2. How do you handle stack overflow and underflow?
Q3. What is the time complexity of stack operations?
Q4. How is stack used in recursion?
Q5. What is a queue?
Q6. What is the FIFO principle?
Q7. What are the basic operations of a queue?
Q8. Difference between queue and stack.
Q9. What are the types of queues?
Q10. How is a queue implemented?
Q11. What is a priority queue?
Q12. What are the applications of queue?
Q13. What is a circular queue?
Q14. Why do we need a circular queue?
Q15. Difference between linear queue and circular queue.
Q16. How do you detect full and empty conditions in a circular queue?
Q17. What is the advantage of circular queue over linear queue?
Q18. How does wrap-around work in circular queue?

MCQ Questions :-

==========

Q1. Which principle is followed by a stack?
A) FIFO
B) LIFO
C) LILO
D) Random Access

Q2. Which operation is used to insert an element into a queue?
A) Pop
B) Push
C) Enqueue
D) Delete

Q3. What will be the output of the following code?

```
Stack<Integer> s = new Stack<>();
s.push(10);
s.push(20);
s.push(30);
System.out.println(s.pop());
```

A) 10

B) 20
C) 30
D) Error

Q4. Which condition represents queue underflow?
A) Queue is full
B) Queue is empty and delete operation is performed
C) Queue has one element
D) Rear reached last index

Q5. What will be the output of the following code?
```
Queue<Integer> q = new LinkedList<>();
q.add(5);
q.add(10);
q.remove();
System.out.println(q.peek());
```
A) 5
B) 10
C) null
D) Error

Q6. In circular queue of size n, the next position of rear is calculated by:
A) rear + 1
B) (rear + 1) % n
C) rear - 1
D) (rear - 1) % n

Q7. What will be the output of the following code?
```
Stack<Character> s = new Stack<>();
s.push('A');
s.push('B');
System.out.println(s.peek());
```
A) A
B) B
C) null
D) Error

Q8. Which data structure is mainly used in Breadth First Search (BFS)?
A) Stack
B) Queue
C) Array
D) Tree

Q9. What will be the output of the following code?

```
Queue<String> q = new LinkedList<>();
q.add("X");
q.add("Y");
q.add("Z");
System.out.println(q.poll());
```

A) X
B) Y
C) Z
D) null

Q10. Which of the following applications uses stack internally?
A) CPU Scheduling
B) Function Calls / Recursion
C) Printer Queue
D) BFS Traversal

Practice Questions :-

==========

Q1. Implement Stack using Array
Write a Java program to implement stack using array. Perform the following operations:

- **Push elements into stack**
- **Pop one element**
- **Display stack elements**

Input:
Enter size: 5
Push: 10, 20, 30
Pop once
Output:
Removed Element: 30
Stack Elements: 10 20

Q2. Implement Queue using Array
Write a Java program to implement queue using array. Perform the following operations:

- **Insert elements into queue**
- **Delete one element**
- **Display queue elements**

Input:
Enter size: 5
Insert: 11, 22, 33
Delete once

Output:
Deleted Element: 11
Queue Elements: 22 33

Q3. Reverse Array using Stack Array Logic
Write a Java program to reverse an integer array using stack implemented with array.
Input:
Array Size: 5
Elements: 1 2 3 4 5
Output:
Reversed Array: 5 4 3 2 1

Q4. Implement Circular Queue using Array
Write a Java program to implement circular queue using array. Perform operations:

- **Insert elements**
- **Delete two elements**
- **Insert new elements using wrap-around**
- **Display final queue**

Input:
Size: 4
Insert: 10, 20, 30, 40
Delete: 2 elements
Insert: 50, 60
Output:
Final Queue: 30 40 50 60

Q5. Check Balanced Parentheses using Character Stack Array
Write a Java program to check whether brackets are balanced or not using stack implemented with character array.
Input:
Expression: {[()]}
Output:
Balanced Expression
Input:
Expression: {[(])}
Output:
Not Balanced Expression